

Idempotency

12 August 2026 10:13 PM

1. What Idempotency Means

Idempotency means:

Repeating the same operation produces the same intended result as doing it once.

Why do we need it?

Networks fail, requests timeout, servers crash, and messages can be delivered more than once. A retry without idempotency can cause duplicate payments, orders, emails, etc.

Examples

Operation	Idempotent?
Set status to ACTIVE	☒ Yes
Delete a session	☒ Yes
Increment a counter	☒ No
Add an item to a list	☒ No
Create payment without an ID	☒ No

Important: Idempotency is about the **final effect**, not necessarily identical responses.

2. Natural Idempotency vs Engineered Idempotency

Natural Idempotency

The operation itself naturally produces the same final state.

Example:

UPDATE users

SET status = 'ACTIVE'

WHERE id = 123;

Running it multiple times still leaves the user:

ACTIVE

Engineered Idempotency

The operation isn't naturally safe, so we give it a unique **operation ID**.

Example:

Add 10 items

Operation ID = shipment_789

The system remembers:

shipment_789 → already processed

So another retry doesn't add another 10 items.

3. Idempotency Keys

An **idempotency key** is a unique ID representing **one logical operation**.

Example:

POST /payments

Idempotency-Key: payment_123

If the request fails because the response was lost, the client retries using the **same key**.

Request 1 → payment_123

Request 2 → payment_123

Request 3 → payment_123

The server knows:

"These are retries of the same payment."

Good Idempotency Keys

A good key should be:

- Client-generated
- Stable across retries
- Unique for different operations
- Scoped to user/tenant/operation
- Connected to the request body
- Stored durably

Never generate the key only after the server receives the request. The client needs the same key when retrying.

4. Server-Side Implementation

The server normally stores idempotency information in a database.

Example:

Idempotency Key



Database



Status



Response

When a request arrives:

Request



Check key



New?



Yes No



Reserve Check existing status

Possible states:

IN_PROGRESS

COMPLETED

Important: Atomic Reservation

Two requests may arrive at exactly the same time.

Bad approach:

Request A → Check key → Doesn't exist

Request B → Check key → Doesn't exist

A → Process payment

B → Process payment **✗**

Instead, reserve the key **atomically** using things like:

- Unique database constraint
- Transaction
- Lock
- Compare-and-set

Status Lifecycle

IN_PROGRESS



COMPLETED

If the server crashes while processing, a locked_until lease can allow another retry to take over after the timeout.

5. Handling In-Progress Requests

Sometimes a retry arrives while the first request is still running.

Example:

Request A → Processing...

Request B → Same idempotency key

The server can:

Strategy	Meaning
409 Conflict	Operation is currently running
202 Accepted	Operation is running; client can check later
Wait briefly	Wait for the first request
Return partial state	Show current workflow state

For long-running operations, **202 Accepted + operation resource** is often useful.

POST /payments



202 Accepted

GET /operations/op_123



```
{  
  "status": "completed"  
}
```

6. External Side Effects

This is one of the hardest idempotency problems.

Example:

1. Reserve idempotency key
2. Save payment attempt
3. Call payment provider
4. Provider charges card
5. Your server crashes **✗**

Now your database doesn't know whether the payment happened.

When the client retries, you **must not blindly charge again**.

Better approach:

Your System



Save payment attempt



Use same idempotency key



Payment Provider



Save provider charge ID

You can also query the provider later to recover the actual state.

Key idea:

A database transaction cannot automatically include an external API.

So you need a **recovery strategy**.

7. Idempotency in Messaging

Message brokers often provide **at-least-once delivery**.

That means the same message may arrive multiple times.

Message: shipment_123

Worker → Process

Worker crashes

Broker → Sends shipment_123 again

The consumer should safely handle this.

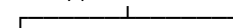
Message



Check operation ID



Already processed?



↓

Skip



ACK

No



Process



Save + ACK

Use durable storage such as:

- Processed-message table
- Business table with unique ID
- Durable key-value store
- State machine

Important: Save the business change before acknowledging the message.

8. HTTP Method Behavior

HTTP methods have different idempotency behavior.

Method	Idempotent?	Example
GET	☒ Yes	Read user
HEAD	☒ Yes	Get headers
PUT	☒ Yes	Replace resource
DELETE	☒ Yes	Delete resource
POST	☒ Usually no	Create payment
PATCH	Depends	Update field

Examples:

PUT /users/123/status

{ "status": "ACTIVE" }

☒ Idempotent.

But:

```
PATCH /inventory/1  
{ "increment_by": 10 }
```

✗ Not idempotent because every retry adds another 10.